CLOUD ENGINEERING BOOTCAMP · DAY 4

Git & GitHub Fundamentals

Track every change, collaborate with confidence, and ship infrastructure the way real engineering teams do.

DURATION

3 hours

PREREQUISITES

Day 1–3: Computers, Linux CLI, and Networking

SESSION OVERVIEW

Learning Objectives

By the end of today, you will be able to:

Explain version control and why Git exists

Create repositories, stage, and commit changes

Create, switch, and merge branches

Resolve a simple merge conflict

Push, pull, and clone with GitHub

Understand Pull Requests and code review

SESSION OVERVIEW

Today's Agenda

1. Why Version Control? Git & GitHub Basics
2. Installing & Configuring Git
3. Repositories, Staging & Commits
4. Branching & Merge Conflicts
5. GitHub: Remotes, Push, Pull & Clone
6. Pull Requests & Team Collaboration
7. Hands-On Lab & Quiz

Let's Warm Up

“What happens if you accidentally delete your project?”

No backup? · Cloud sync? · A friend's copy? · “I'd just... start over”

Life Without Version Control

OBJECTIVE: Recognize the everyday problems Git was built to solve.

Losing Files

Multiple Copies

No History

Team Conflicts

A FAMILIAR NIGHTMARE

```
Project_v1.zip
  Project_v2.zip
    Project_Final.zip
      Project_Final_Final.zip
        Project_Final_REAL.zip
```

Sound familiar? Git replaces this entire mess with one clean, trackable history.

What Is Git?

OBJECTIVE: Understand Git as a distributed version control system built on snapshots.

Git is a distributed version control system — it records snapshots of your project over time inside a repository.

- Tracks every change, forever
- Works fully offline (local-first)
- Every teammate has a full copy of the history

REAL-WORLD ANALOGY: *Like a video game's save-point system — you can always return to any earlier save.*

What Is GitHub?

OBJECTIVE: Distinguish Git (the tool) from GitHub (the cloud platform built around it).

Cloud Hosting

Stores your repositories online

Collaboration

Teams work on the same code together

Portfolio

Public profile of your projects

Open Source

Millions of public projects to explore

Git = the tool that runs on your computer. **GitHub** = a website that hosts Git repositories in the cloud.

Installing Git

OBJECTIVE: Get Git running on your machine, whatever OS you use.

Windows

Download Git for Windows from git-scm.com

macOS

Run: xcode-select --install (or use Homebrew)

Linux

Run: sudo apt install git (Debian/Ubuntu)

Verify Installation

```
$ git --version
git version 2.43.0
```

Git Configuration

OBJECTIVE: Tell Git who you are — every commit is signed with this identity.

user@cloud:~/project

```
$ git config --global user.name "Alex Rivera"
$ git config --global user.email "alex@example.com"
```

WHY IT MATTERS

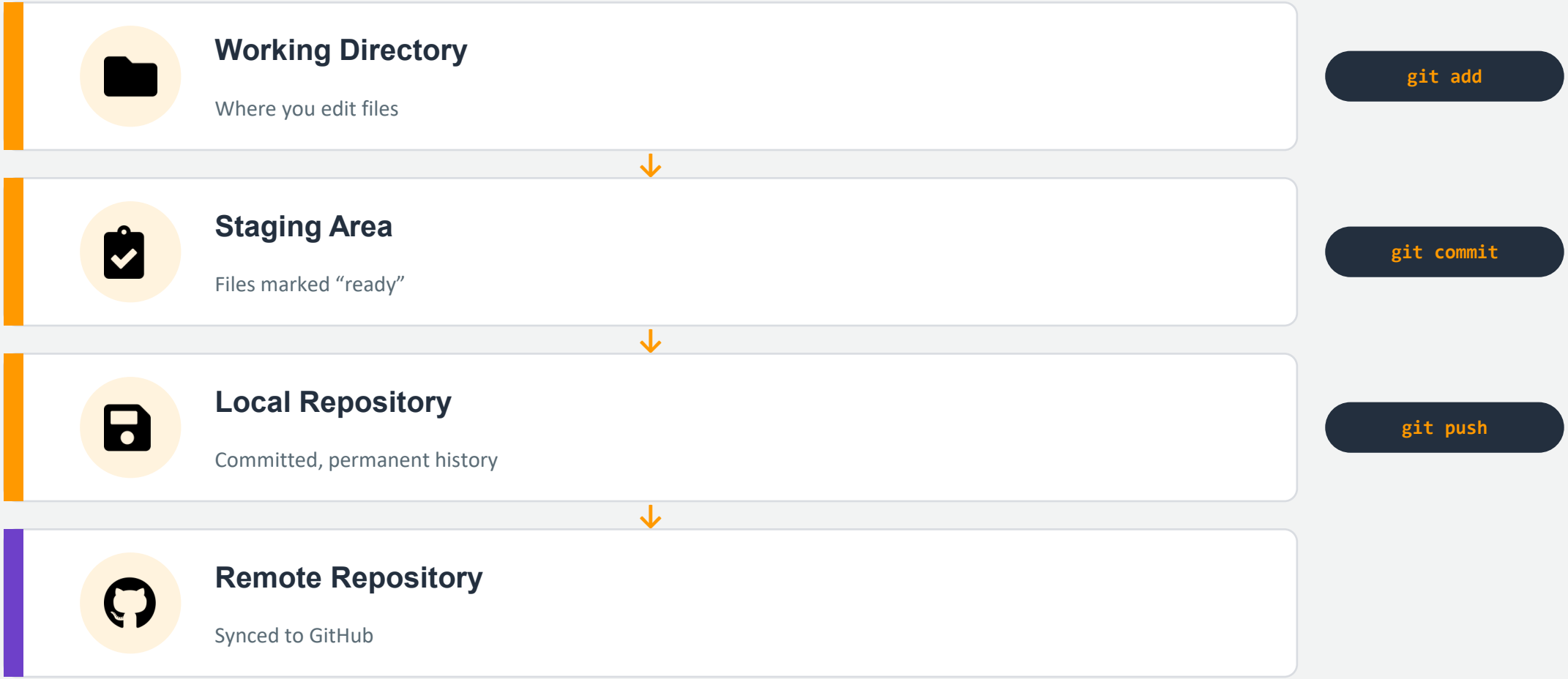
This name and email get permanently attached to every commit you make — it's how your team knows who changed what.

REAL-WORLD ANALOGY: *Like signing your name on a shared whiteboard — everyone can see exactly who wrote what, and when.*

INSTRUCTOR TIP: Use the same email here that you'll use for your GitHub account — it links your commits to your GitHub profile automatically.

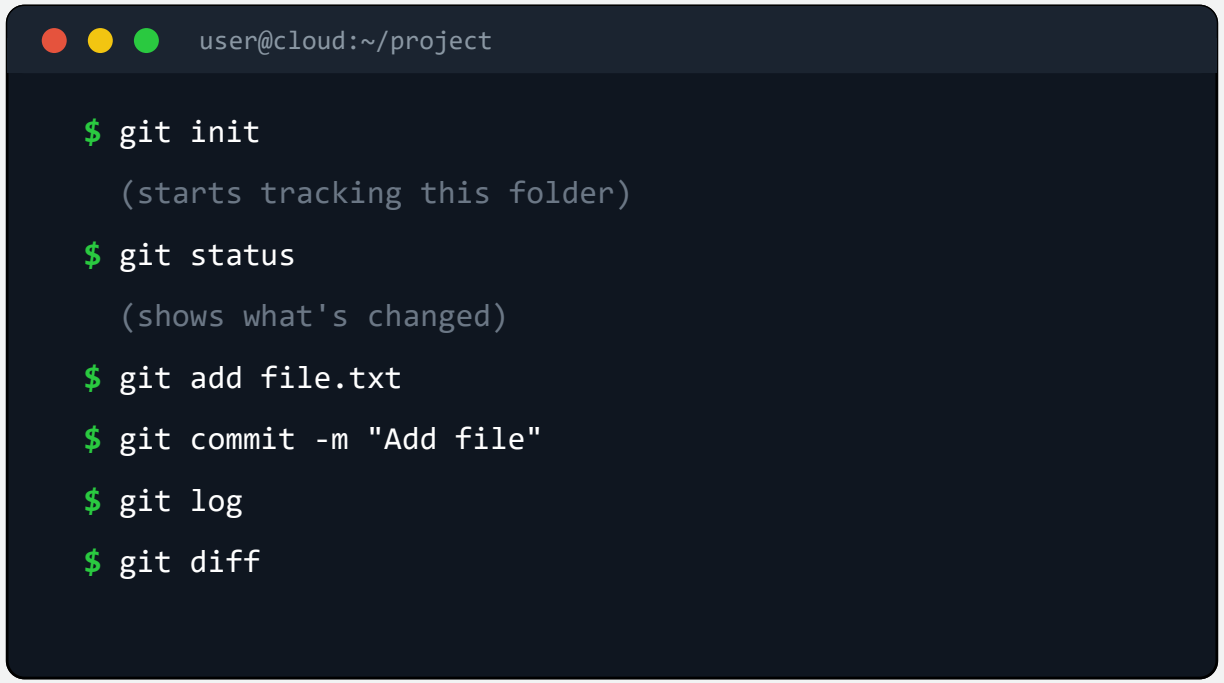
The Git Workflow

OBJECTIVE: See the four stages a file passes through as you save your work.



Essential Git Commands

OBJECTIVE: Six commands you'll type constantly, starting today.



`git init`

Turn a folder into a Git repository

`git status`

See what's changed right now

`git add`

Stage a file for the next commit

`git commit`

Save a permanent snapshot

`git log`

View commit history

`git diff`

See exact line-by-line changes

First Repository — Guided Lab

OBJECTIVE: Create your very first Git repository from scratch.

First Repository Lab

```
$ mkdir my-first-repo && cd my-first-repo
$ git init
Initialized empty Git repository
$ echo "# My First Repo" > README.md
$ git status
Untracked files: README.md
$ git add README.md
$ git commit -m "Initial commit"
1 file changed, 1 insertion(+)
```

Understanding Commits

OBJECTIVE: Write commit messages your future self (and teammates) will thank you for.

GOOD

```
"Fix login redirect bug for expired
sessions"
```

Specific, clear, describes the why

BAD

```
"fixed stuff" / "asdf" / "final version 2"
```

Vague — tells you nothing later

REAL-WORLD ANALOGY: *A commit is like a diary entry with a photo attached — the message is the caption explaining what changed and why.*

INSTRUCTOR TIP: Write commit messages in the present tense, e.g. “Add login validation”, not “Added” or “Adding” — it's the Git community standard.

Git Branching

OBJECTIVE: Understand how branches let you experiment without touching working code.

main branch

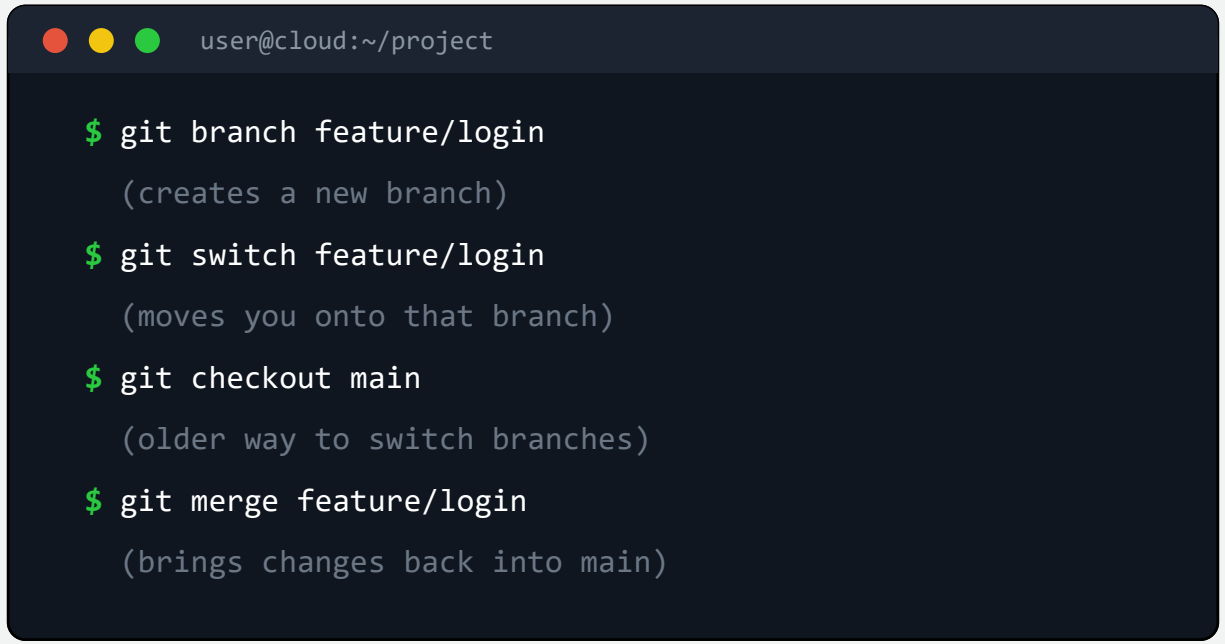
The stable, always-working version of your project.

feature branch

An isolated copy to build or test something new, safely.

Working with Branches

OBJECTIVE: Create, switch between, and merge branches with four commands.



`git branch`

List or create branches

`git switch`

Move to a different branch (modern)

`git checkout`

Switch branches (classic / also restores files)

`git merge`

Combine another branch into the current one

Merge Conflicts

OBJECTIVE: Understand why conflicts happen and resolve one step by step.

Why they happen

Two branches changed the exact same line of the same file, and Git can't decide which version is correct.

index.html — conflict markers

```
<<<<<<< HEAD
Welcome to our site!
=======
Welcome to my site!
>>>>>>> feature/login
```

RESOLUTION STEPS

1. Open the file & find the <<<< ==== >>>> markers
2. Decide which lines to keep (or blend both)
3. Delete the marker lines entirely
4. git add the file, then git commit

INSTRUCTOR TIP: Merge conflicts are completely normal, even for senior engineers — they're not a sign something went wrong.

Remote Repositories

OBJECTIVE: Connect your local repository to a copy hosted on GitHub.


```
user@cloud:~/project

$ git remote add origin https://github.com/you/my-first-repo.git
$ git remote -v
```

GitHub Walkthrough

OBJECTIVE: Create an account, a repository, and push your first project.

1

Create a free account

`github.com → Sign Up`

2

Create a new repository

`Click “New” → name it → Create`

3

Push your project

`git push -u origin main`

4

View commit history

`Repo page → “Commits” tab`

Clone vs. Fork

OBJECTIVE: Know which one to use depending on whether you own the repository.

Clone

Downloads a full copy of a repo you already have access to — usually your own team's project.

```
git clone <url>
```

Use when: you're a collaborator on the project.

Fork

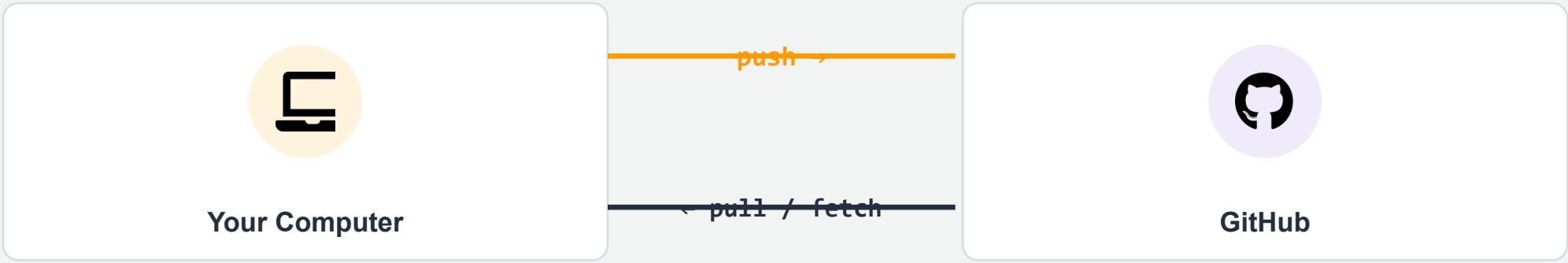
Creates your own copy of someone else's repository, under your account, that you fully control.

```
Click “Fork” on GitHub
```

Use when: contributing to open-source you don't own.

Push and Pull

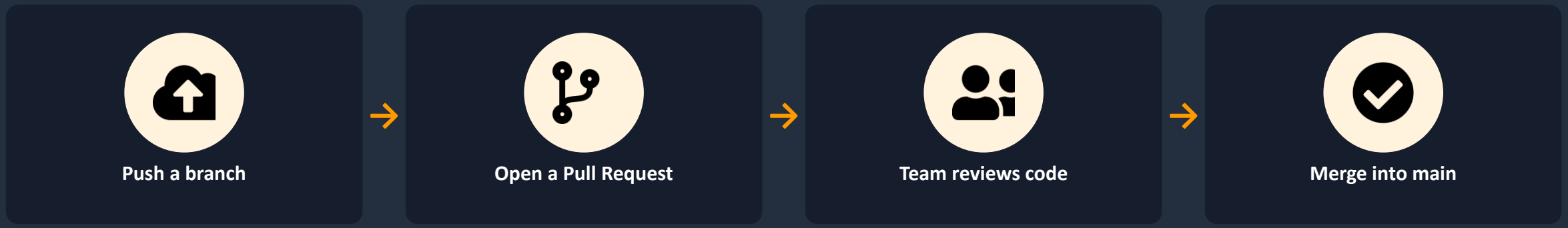
OBJECTIVE: Keep your local and remote repositories in sync.



`git push`	Upload your local commits to GitHub
`git pull`	Download & merge remote changes into your local branch
`git fetch`	Download remote changes without merging yet

Pull Requests

OBJECTIVE: Understand how teams review and approve code before it merges.



- Catches bugs before they reach main
- Shares knowledge across the team
- Creates a documented discussion trail

REAL-WORLD ANALOGY: *Like submitting a manuscript to an editor before it's published — a second set of eyes catches what you might miss.*

WHY THIS MATTERS

GitHub for Cloud Engineers

OBJECTIVE: See exactly what cloud engineers store in Git, every single day.

Terraform

CloudFormation

Python Scripts

Documentation

CI/CD Pipelines

Config Files

BEST PRACTICES

GitHub Best Practices

Meaningful Commits

Describe the why, not just the what

Small Commits

One logical change per commit

Branch Naming

feature/, fix/, chore/ prefixes

README Files

Explain what the project does & how to run it

.gitignore

Keep secrets & junk files out of Git

Pull Before Push

Stay in sync, avoid conflicts

Put It All Together

Twelve steps, from git init to a clone on a second machine.

HANDS-ON LAB

Lab Steps — Part 1: Local Git

1

```
git --version
```

Confirm Git is installed

2

```
git config --global user.name "You"
```

Set your identity (if not already done)

3

```
mkdir lab-project && cd lab-project && git init
```

Create & initialize a new repository

4

```
echo "# Lab Project" > README.md
```

Create a README file

5

```
git add README.md && git commit -m "Initial commit"
```

Stage and commit your first snapshot

6

```
git switch -c feature/add-notes
```

Create and switch to a feature branch

HANDS-ON LAB

Lab Steps — Part 2: GitHub Sync

7

```
echo "- Learned branching" >> README.md
```

Add a new line to README on your feature branch

8

```
git add . && git commit -m "Add notes to README"
```

Commit the change

9

```
git switch main && git merge feature/add-notes
```

Merge the feature branch back into main

10

```
Create a repo on github.com, then:
git remote add origin <url>
```

Connect your local repo to GitHub

11

```
git push -u origin main
```

Push your commits to GitHub

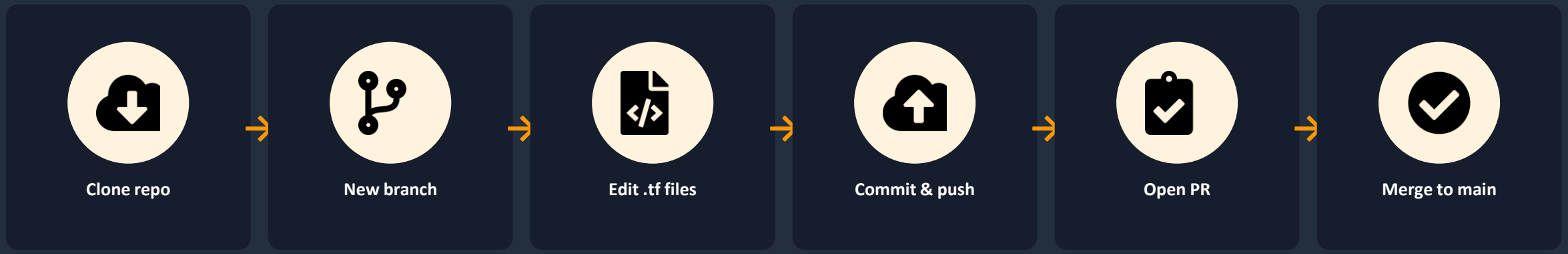
INSTRUCTOR TIP: If push asks for a password and fails, you likely need a Personal Access Token — see the Appendix for setup steps.

Clone the repo elsewhere, then pull the latest

REAL-WORLD SCENARIO

A Team Shipping Terraform

OBJECTIVE: See today's full Git workflow play out on a real cloud engineering team.



Three engineers, one shared AWS infrastructure repo — each on their own branch, reviewing each other's Terraform changes before anything reaches production.

INSTRUCTOR TIP: This exact workflow — branch, PR, review, merge — is how real infrastructure changes get made safely at almost every company.

Avoid These Early Pitfalls

Forgetting git add

Editing files but never staging them before commit

Committing Secrets

Accidentally pushing API keys or passwords

Working Directly on main

Making risky changes with no safety branch

Poor Commit Messages

“fix” or “update” tell nobody anything useful

Fear of Merge Conflicts

Avoiding branches entirely to dodge conflicts

Not Pulling First

Pushing without syncing, causing avoidable conflicts

WRAP-UP

What We Covered Today

- Version control solves lost files, no history, and team conflicts
- Git tracks snapshots locally: add → commit → log
- Branches let you experiment safely; merges bring work back together
- GitHub hosts your repos in the cloud: push, pull, clone, fork
- Pull Requests add review before code reaches main
- This exact workflow powers Terraform, CI/CD, and every DevOps pipeline

CHECK-IN

Quiz — Questions 1–5

1. What problem does version control primarily solve?
2. What is the difference between Git and GitHub?
3. Which command turns a folder into a Git repository?
4. What does git add actually do?
5. What does a commit represent?

CHECK-IN

Quiz — Questions 6–10

6. What is the purpose of a feature branch?

7. What causes a merge conflict?

8. What does git push actually do?

9. What's the difference between git pull and git fetch?

10. When would you fork a repository instead of cloning it?

CHECK-IN

Quiz — Questions 11–15

11 What is a Pull Request, and why does it matter?

12 Name two things a .gitignore file should typically exclude.

13 Why is 'fixed stuff' considered a bad commit message?

14 Why do cloud engineers store Terraform code in Git?

15 What should you do if you accidentally push a secret to GitHub?

CHECK-IN

Quiz — Answer Key

1. Losing files, no history, and conflicting copies

2. Git = local tool; GitHub = cloud hosting for Git repos

3. git init

4. Stages a file, marking it ready for the next commit

5. A permanent, labeled snapshot of your project

6. Build/test something new without touching main

7. Two branches changed the same line of the same file

8. Uploads your local commits to GitHub

9. pull merges automatically; fetch downloads only

10. When you don't own / can't write directly to the repo

11. A reviewed request to merge a branch into main

12. Secrets (.env), build folders, OS files (e.g. .DS_Store)

13. It doesn't explain what changed or why

14. It's text — trackable, reviewable, revertible like code

15. Rotate the credential immediately, then clean history

BEFORE DAY 5

Homework

Build a Portfolio Repo

Create a new GitHub repository with a README describing yourself as a cloud engineer.

Practice Branching

Create 2 feature branches, make changes on each, and merge both into main.

Cause & Fix a Conflict

Intentionally edit the same line on two branches, then resolve the conflict.

Reference & Cheat Sheets

Quick-reference material to keep handy as you continue building with Git.

Git Cheat Sheet

`git init` Start tracking a folder	**`git status`** See what's changed	**`git add <file>`** Stage a file
`git commit -m "msg"` Save a snapshot	**`git log`** View commit history	**`git diff`** See line-by-line changes
`git branch` List / create branches	**`git switch <b>`** Change branches	**`git merge <b>`** Combine a branch in
`git remote -v` Show connected remotes	**`git push`** Upload commits	**`git pull`** Download & merge changes

GitHub Cheat Sheet

`git clone <url>`

Download a repo you have access to

`Fork (on GitHub)`

Copy someone else's repo to your account

`git remote add origin <url>`

Link a local repo to GitHub

`git push -u origin main`

First push, sets tracking

`Pull Request`

Propose merging a branch, with review

`Issues tab`

Track bugs & feature requests

`Actions tab`

Automated CI/CD workflows

`Settings → Tokens`

Create a Personal Access Token

Branching Workflow & Conflict Guide

BRANCHING WORKFLOW

1. Start from an up-to-date main (git pull)
2. Create a branch: git switch -c feature/x
3. Make small, focused commits
4. Push the branch: git push -u origin feature/x
5. Open a Pull Request on GitHub
6. Address review feedback
7. Merge, then delete the branch

MERGE CONFLICT RESOLUTION

1. Run git status to find conflicted files
2. Open the file, find <<<< ==== >>>> markers
3. Decide what the final content should be
4. Delete all marker lines completely
5. git add the resolved file
6. git commit to complete the merge
7. Push, then confirm on GitHub

Useful Shortcuts & Commands

`Ctrl + C`

Cancel a running terminal command

`Tab`

Auto-complete file & branch names

`↑ / ↓ arrows`

Cycle through command history

`git log --oneline`

Compact one-line commit history

`git branch -d <name>`

Delete a merged branch

`git checkout -- <file>`

Discard local changes to a file

INSTRUCTOR TIP: Tab-completion alone will save you dozens of typos a day — build the habit early.

Great Work Today!

You can now track changes, branch, merge, and collaborate with Git and GitHub — just like a real engineering team.